

Document Number: P3167R0
Date: 2024-02-28
Audience: SG21
Reply-to: Tom Honermann <tom@honermann.net>

Attributes for the result name in a postcondition assertion

- [Introduction](#)
- [Motivation](#)
- [Proposal](#)
- [Implementation experience](#)
- [Acknowledgements](#)
- [References](#)
- [Wording](#)

Introduction

SG21 confirmed support for [P3088R1 \(Attributes for contract assertions\)](#) [\[P3088R1\]](#) during its [2024-02-15 telecon](#) and the approved changes were subsequently added to [P2900R5 \(Contracts for C++\)](#) [\[P2900R5\]](#). This established syntactic locations for attributes that appertain to contract assertions and the assertion statement as follows:

```
int f(int i, int j)
  pre [[foo]] (i > 0 && j > 0)
  post [[bar]] (r: r > 0)
{
  [[Likely]] contract_assert [[baz]] (i + j < 10);
  return i + j;
}
```

However, P3088R1 did not provide for an attribute to appertain to the result name optionally declared in a postcondition specifier; the **r** declared in the example postcondition above. This proposal seeks to fill that gap by allowing an attribute to be specified in the result name introducer.

```
int g()
  post (r [[maybe_unused]]: r > 0);
```

Motivation

As evidenced by [annex A \(Grammar summary\)](#) in the C++ standard, an attribute may appertain to almost every kind of declared name or entity including:

- Type aliases (when declared with an *alias-declaration*)
- Enumerations
- Enumerators
- Variables (including the unnamed object for structured bindings)
- Structured bindings (if [P0609R2](#) [\[P0609R2\]](#) is adopted)

- Classes
- Data members
- Functions
- Parameters (function parameters, non-type template parameters, exception handlers)
- Concepts
- Labels
- Namespaces
- Modules

Exceptions include:

- Namespace aliases
- Type template parameters
- Template template parameters
- Lambda captures

It is not clear why attributes are not supported for the items in the exceptions list. The recommended practice for the `[[deprecated]]` attribute in [\[dcl.attr.deprecated\]p4](#) seems applicable to namespace aliases:

Recommended practice: Implementations should use the deprecated attribute to produce a diagnostic message in case the program refers to a name or entity other than to declare it, after a declaration that specifies the attribute. The diagnostic message should include the text provided within the [attribute-argument-clause](#) of any deprecated attribute applied to the name or entity. The value of a [has-attribute-expression](#) for the deprecated attribute should be `0` unless the implementation can issue such diagnostic messages.

Likewise, the recommended practice for the `[[maybe_unused]]` attribute in [\[dcl.attr.unused\]p4](#) seems applicable to the other entities:

Recommended practice: For an entity marked `maybe_unused`, implementations should not emit a warning that the entity or its structured bindings (if any) are used or unused. For a structured binding declaration not marked `maybe_unused`, implementations should not emit such a warning unless all of its structured bindings are unused. For a label to which `maybe_unused` is applied, implementations should not emit a warning that the label is used or unused. The value of a [has-attribute-expression](#) for the `maybe_unused` attribute should be `0` if the attribute does not cause suppression of such warnings.

The `[[deprecated]]` attribute was adopted for C++14 via [N3760](#) [\[N3760\]](#). Neither that paper, its previous revision, or the minutes from EWG and CWG review during the following meetings demonstrate consideration for use of the attribute with a namespace alias.

- [EWG review of N3394 in Portland, 2012](#)
- [CWG review of N3760 in Chicago, 2013](#)

The `[[maybe_unused]]` attribute was adopted for C++17 via [P0212R1](#) [\[P0212R1\]](#). Neither that paper, its previous revision, the rationale provided in the previous [P0068R0](#) [\[P0068R0\]](#), or the minutes from EWG and CWG review during the following meetings demonstrate consideration for use of the attribute with a type template parameter, template template parameter, or lambda capture.

- [EWG review of P0068R0 in Kona, 2015](#)
- [CWG review of D0212R1 in Jacksonville, 2016](#)

A search of both active and closed CWG issues failed to identify any existing issues that concern attribute appertinment for the excluded items.

There does not appear to be a recorded rationale for omitting attribute support for the excluded items above.

Absent compelling rationale otherwise, precedent indicates that new kinds of declared names or entities should allow for attribute appertainment and their respective declarations should therefore include syntactic locations for an attribute specifier sequence.

Proposal

- Allow *attribute-specifier-seq* to optionally follow *identifier* in a *result-name-introducer* with the effect that the specified attributes (if any) appertain to the declared result name.
- Allow the `[[maybe_unused]]` attribute to appertain to a result name.

Implementation experience

None. The experimental implementation of [P2900R5](#) [\[P2900R5\]](#) in gcc hosted on <https://godbolt.org> does not currently diagnose an unused result name, nor does it allow an attribute specifier sequence to appear in a *result-name-introducer* as demonstrated at <https://godbolt.org/z/339WEr7c3>.

Acknowledgements

Thanks to Timur Doumler and Joshua Berne for their previous work to enable attributes for contract assertions in [P3088R1](#) [\[P3088R1\]](#) and for the tremendous amount of time, energy, and expertise they have each contributed to SG21 and advancing contracts for C++.

Additional thanks to Timur Doumler for suggesting much of the wording included in this proposal.

References

- [N3760] "[[deprecated]] attribute", N3760, 2013.
<https://wg21.link/n3760>
- [P0068R0] "Proposal of [[unused]], [[nodiscard]] and [[fallthrough]] attributes.", P0068R0, 2015.
<https://wg21.link/p0068r0>
- [P0212R1] "Wording for [[maybe_unused]] attribute.", P0212R1, 2016.
<https://wg21.link/p0212r1>
- [P0609R2] "Attributes for Structured Bindings", P0609R2, 2023.
<https://wg21.link/p0609r2>
- [P2900R5] "Contracts for C++", P2900R5, 2024.
<https://wg21.link/p2900r5>
- [P3088R1] "Attributes for contract assertions", P3088R1, 2024.
<https://wg21.link/p3088r1>

Wording

These changes are relative to [P2900R5](#) [\[P2900R5\]](#) and [P0609R2](#) [\[P0609R2\]](#).

- ☐ Hide inserted text
- ☐ Hide deleted text

Modify [dcl.contract.func]:

Drafting note: This depends on the addition of *attributed-identifier* from [P0609R2](#) [\[P0609R2\]](#).

```

function-contract-specifier-seq :
    function-contract-specifier function-contract-specifier-seq

function-contract-specifier :
    precondition-specifier postcondition-specifier

precondition-specifier :
    pre attribute-specifier-seqopt ( conditional-expression )

postcondition-specifier :
    post attribute-specifier-seqopt ( result-name-introduceropt conditional-expression )

result-name-introducer :
    attributed-identifier

```

A *function contract assertion* is a contract assertion ([basic.contract]) associated with a function. Each *function-contract-specifier* of a *function-contract-specifier-seq* (if any) of an unspecified first declaration of a function introduces a corresponding function contract assertion for that function. The optional *attribute-specifier-seq* following *pre* or *post* appertains to the introduced contract assertion. The optional *attribute-specifier-seq* of the *attributed-identifier* in a *result-name-introducer* appertains to the introduced result name. [*Note:* The *function-contract-specifier-seq* of a *lambda-declarator* applies to the call operator or operator template of the corresponding closure type ([expr.prim.lambda.closure]). — *end note*]

[...]

Add a proposed modification to [\[dcl.attr.unused\]](#), paragraph 2:

Drafting note: This includes [modifications from P0609R2](#) [\[P0609R2\]](#).

The attribute may be applied to the declaration of a class, *typedef-name*, variable (including a structured binding declaration), [structured binding](#), [result name](#), non-static data member, function, enumeration, or enumerator, or to an *identifier* label ([\[stmt.label\]](#)).